Object Oriented Programming Using C++

Ketan Kore

Ketan.Kore@sunbeaminfo.com

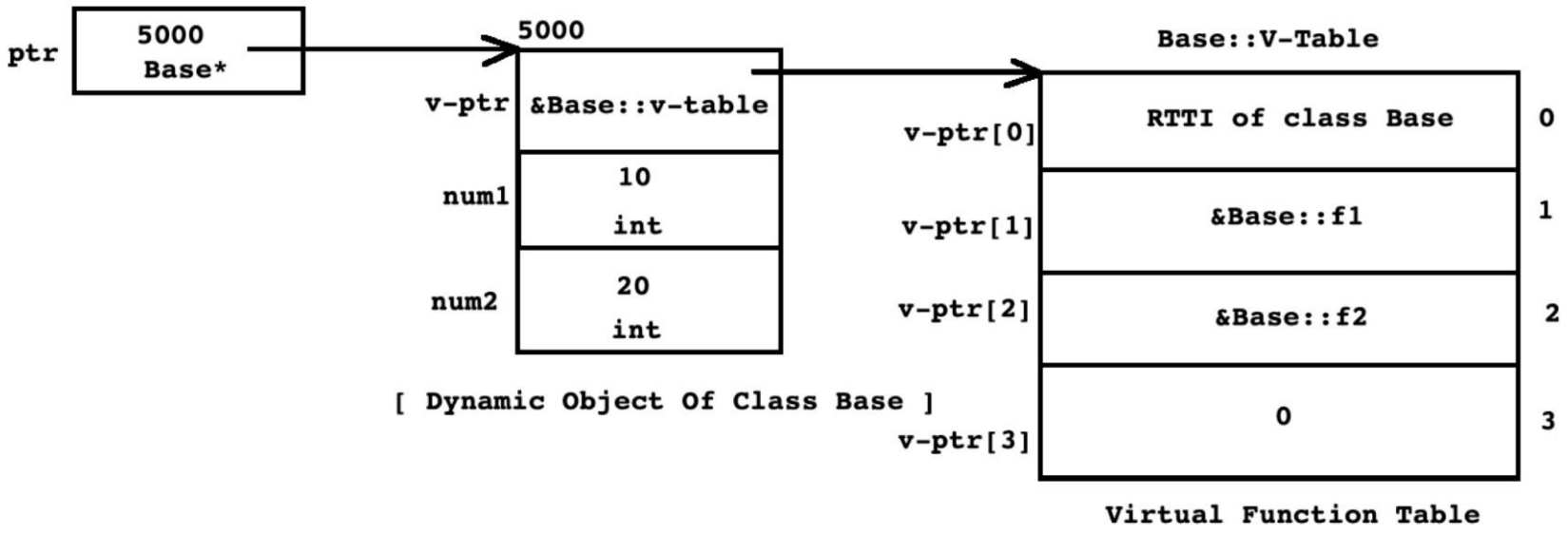
Virtual Function Table and Pointer

- Virtual Function Table
  - If we declare function virtual then compiler implicitly creates one table(array/structure) to store address of that virtual function. Such table is called virtual function table.
  - It is also called as vf-table / v-table.
  - In short, a table which stores address of virtual functions declared inside class is called v-table.
  - Compiler generates v-table once per class.
- Virtual Function Pointer
  - To store address of virtual function table, compiler implicitly declare one pointer as a data member inside class. Such data member is called virtual function pointer/vf-pointer/v-ptr.
  - In short, a pointer which stores address of virtual function table is called v-ptr.
  - Compiler generates v-ptr once per object.

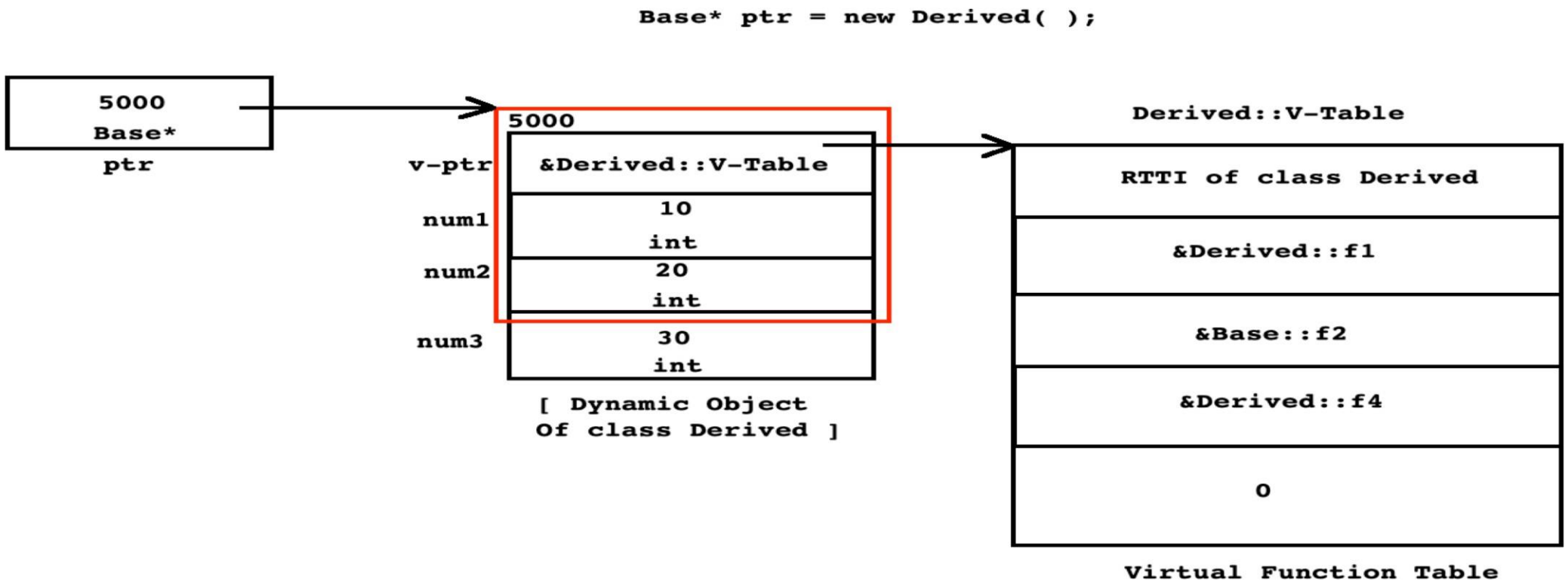
- Size of object of derived class = size of all the non static data member declared in base class + size of all the non static data member declared in derived class + 2/4/8 bytes (If base class / derived class contains at least one virtual function).

Virtual Function Table and Pointer

```
Base *ptr = new Base( );
```



Virtual Function Table and Pointer



Virtual Function

- Why we can not declare constructor virtual?
  - Virtual function is designed to call on base class pointer/reference
  - We can not call constructor on object/pointer/reference explicitly. It is designed to call implicitly.
  - Since we can not call constructor on pointer/reference, we can not declare it virtual.
- In case of upcasting, to call destructor of derived class, it is necessary to declare destructor in base class virtual.
- Points to remember
  1. According to problem statement / client's requirement, if implementation of base class member function is logically 100% complete(no need to override) then we should declare it non virtual.
  2. According to problem statement / client's requirement, if implementation of base class member function is logically incomplete / partially complete (need to override) then we should declare it virtual.
  3. According to problem statement / client's requirement, if implementation of base class member function is logically 100% incomplete(must override) then we should declare it pure virtual.

Friend Function and Friend Class

- If we want to access private members outside class then we should use:
    1. Member function(Getter and Setter)
    2. Friend function
    3. Pointer
- A non member function of the class, which is designed to access private and protected members of the class is called friend function of the class.
- friend is a keyword in C++.
- We can write friend declaration statement inside any section(private/protected/public) of the class.
- We can write friend declaration statement inside class only.
- Including main function, we can declare any global function friend.
- If we declare function as a friend inside class then function is not considered as a member of a class.
- We can declare single function as a friend into multiple classes.

Friend Function and Friend Class

- We can declare member function of a class as a friend inside another class.

```
class A{
public:
    void sum( void );
};
class B{
private:
    int num1;
    int num2;
public:
    B( void );
    friend void A::sum( void );
};
```

Friend Function and Friend Class

- If we want to access private and protected members of the class inside some of the member functions of another class then we should declare member function friend.
- If we want to access private and protected members of the class inside all of the member functions of another class then we should declare class friend.
- If we want to declare class/function as a friend inside another class then function and class must be exist inside same namespace.

Exception Handling

- Types of Error

1. **Compiler error**
   - It gets generated due to syntactical mistake.
2. **Linker error**
   - If we try to use any variable/function without definition then we get linker error.
3. **Bug**
   - Logical error is also called bug.
4. **Runtime error**
   - It gets generated due to wrong input.

Exception Handling

- **Operating System Resources**
    1. Memory
    2. File
    3. Thread
    4. Socket
    5. Connection
    6. Hardware resources(CPU, Keyboard, Mouse, Monitor)
    7. System call
- Since operating system resources are limited, we should use it carefully. In other words, we should avoid their leakage.

Exception Handling

- **Definition**

    1. Runtime error is also called as exception.
    2. Exception is an object which is used to send notification to the end user of the system if any exceptional situation occurs in the program.

- **Need of exception handling**

    1. If we want to manage OS resources carefully in the program we should handle exception.
    2. If we want to handle all the runtime errors at single place so that we can reduce maintenance of the application.

- **How to handle exception**

    - In C++, we can handle exception using 3 keywords:

        1. Try
        2. Catch
        3. throw

Exception Handling

- **try**
    1. try is a keyword in C++.
    2. If we want to keep watch on group of statements then we should use try block.
    3. try block is also called as try handler.
    4. We can not define try block after catch block/handler.
    5. try block must have at least one catch block.
- **throw**
    1. throw is keyword in C++.
    2. Exception can be raised implicitly or we can generate it explicitly.
    3. If we want to generate exception explicitly then we should use throw keyword.
    4. throw statement is a jump statement
    5. Jump statements in C/C++ : break, return, goto, continue, throw

Exception Handling

- **catch**

    1. catch is keyword in C++.
    2. If we want to handle exception thrown from try block then we should use catch block.
    3. Catch block is also called as catch handler.
    4. Single try block may have multiple catch blocks.
    5. We can not define catch block before try block/handler.
    6. For thrown exception, if matching catch block is not available then C++ runtime environment implicitly give call to the std::terminate() function which implicitly calls std::abort( )function.
    7. A catch block, which can handle all type of exception is called default/generic catch block. E.g. **catch(. . .){ }**
    8. We must define generic catch block after all specific catch block.

Thank you